Full-Stack Blueprint Book

How the portfolio was thought out, built and tested — and how to repeat the method

Version 1.0 · October 2026

Laravel 12 · React · MySQL · GSAP · Sneat admin

Contents

Contents 2

1. What we built — and why this book exists 4

The system in one picture . . . 4

2. The thinking process (10 questions) 6

Why this order? . . . 6

3. The exact sequence I followed 7

4. Key decisions (and the alternatives I rejected) 9

5. Data model 10

6. API design 12

Rules . . . 12

7. The admin dashboard 13

7.1 Sneat inside Laravel . . . 13

7.2 The generic CRUD engine . . . 13

7.3 The rich editor — how "image anywhere, any alignment" works . . . 13

7.4 Live search, settings, messages . . . 14

8. The React frontend 15

8.1 Structure . . . 15

8.2 Provider stack (outer → inner) . . . 15

8.3 Theming . . . 15

8.4 The animation catalogue . . . 15

8.5 The pattern that keeps GSAP safe in React . . . 16

8.6 How the hero portrait works . . . 16

8.7 Performance and accessibility . . . 17

9. Testing — and the 9 real bugs it found 18

10. Four flows, step by step 20

11. Recipes: extend the project 21

Recipe 1 — add "Testimonials" (30 minutes) . . . 21

Recipe 2 — more . . . 21

12. The Playbook — if you build another full-stack project 22

Ten rules I always follow . . . 22

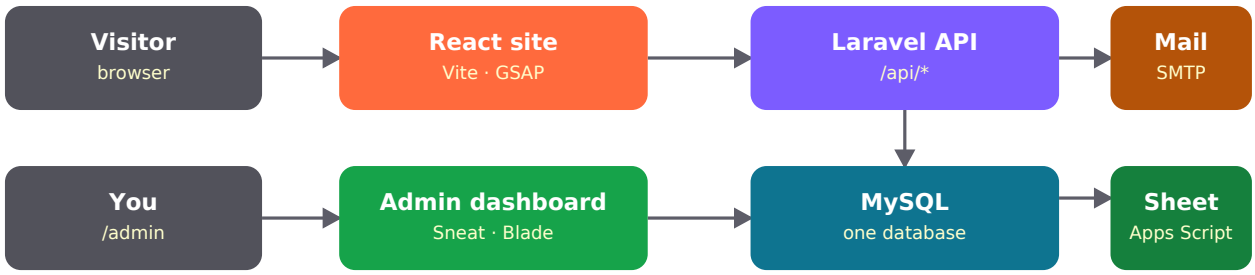
Questions to ask the client in Discovery . . . 23

Common mistakes 23
Git and delivery rhythm 23
Appendix **24**
Glossary (Banglish) 24
Command cheat sheet 24
Where to go next 24

1. What we built — and why this book exists

This is not only a manual. It is the thinking behind the project: what I asked first, what I built first, what I postponed, and how you can repeat the same method on your next full-stack job.

> **Banglay shar kotha**
>
> Ekta full-stack project mane kebol code na — prothome "keno, ke use korbe, ki lagbe" bujhte hoy, tarpor data, tarpor API, tarpor screen. Ei boi te ami jei order e kaj korechi shei order, karon shoho, ar ei sequence ta tumi je kono notun project e abar bebohar korte parbe.



The system in one picture

Three programs, one database. The **React site** is only a skin: it asks the API for content and animates it. The **Laravel app** owns the rules (validation, permissions, e-mail, search) and also renders the **admin dashboard**. **MySQL** is the single source of truth, so a change in admin is visible on the site in the same second.

Requirement you gave	Where it lives	Main files
React site with GSAP, ScrollTrigger, smooth scroll, dark/light, colour settings, responsive	frontend	`context/*`, `hooks/useGsap.js`, `styles.css`
Laravel API + MySQL	backend	`routes/api.php`, `PublicController`, migrations
Admin dashboard in Sneat	backend	`resources/views/admin/*`, `public/sneat`
Rich editor, images left/right/centre at start/middle/end	admin	TinyMCE in `admin-assets/admin.js`, `.prose` CSS
Huge project information	admin + API	`ProjectController` fields, `projects.content` LONGTEXT
Contact → e-mail, Excel / Google Sheet records	backend	`PublicController::contact`, `GoogleSheet`, `XlsxWriter`

Requirement you gave	Where it lives	Main files
Colours manageable from admin	both	`theme_colors` table → `/api/bootstrap` → ⚙ drawer
Live search with image on the right	both	`SearchService`, `SearchOverlay.jsx`, admin navbar
Seed demo blogs / projects / images / videos	backend	`PortfolioSeeder`, `seeders/assets`
Animations everywhere	both	GSAP in React; GSAP + CSS in admin

2. The thinking process (10 questions)

Before touching the keyboard I ask ten questions, always in this order. Each answer becomes a deliverable. If an answer is vague, I stop and clarify — vague answers become expensive bugs three weeks later.

#	Question	Deliverable	In this project
1	**Why** does it exist?	one-sentence goal	A portfolio that wins clients and that its owner can update without a developer
2	**Who** uses it?	user list	Visitors / clients · the owner (admin)
3	**What** must it do? (must / should / later)	feature list	Pages, search, contact pipeline, editor, colours — "later": comments, i18n, 2FA
4	What **data** exists?	entities + relations	projects, posts, categories, messages, settings, media…
5	What is the **contract** between screens and data?	API list + JSON shapes	`/api/bootstrap`, `/projects`, `/posts`, `/search`, `/contact`
6	What does **each screen** show?	sitemap + wireframes	Home, About, Projects, Blogs, Contact, 2 detail pages (design chosen first)
7	How is content **managed**?	admin spec	Generic CRUD engine + settings schema + media library
8	What gives it **feel**?	motion + theming plan	GSAP scrub/3D/ring/curtain, accent colours, motion on/off
9	How do I **prove** it works?	test list	API + admin + browser-like render + MySQL run
10	How is it **shipped & kept alive**?	deploy + backup plan	Nginx/cPanel steps, .env, backups, update routine

Why this order?

- **Data before screens.** A screen is just a view of data. If the data model is right, screens are cheap; if it is wrong, every screen is rewritten.
- **Contract before code.** Writing the JSON shape first lets backend and frontend be built independently, and even tested separately.
- **Admin early, not last.** Without admin you cannot enter real content, so you never see real problems (long titles, 30 images). I built admin before polishing the site.
- **Motion last.** Animation is paint. It must sit on top of working content, never carry it. That is why the site also works with "Motion off".

> **The 80/20 rule used everywhere**
>
> Build the boring 80% generically (one CRUD engine, one settings schema) and spend the saved time on the 20% users actually feel (editor, search, motion, details).

3. The exact sequence I followed

Every step ended with something runnable. I never stacked more than one hour of untested work.

Step	What I did	Why at this moment	Output you can check
1	Designed the **front-end look first** as single HTML files (6 iterations, you chose the final one)	Design decides which data and which pages exist. Cheapest place to change your mind	A page you can open and judge
2	Wrote the **content inventory**: every text/image/list on every page	Each item is either static or comes from the DB — this list becomes the tables	Checklist of fields
3	Prepared the **environment**: Laravel 12 skeleton, MySQL, `.env`	So every later step can be run immediately	`php artisan --version` works
4	**Migration + models** (12 tables)	Data first	`migrate:fresh` succeeds
5	**Public API** (bootstrap, projects, posts, search, contact)	Contract first; unlocks the React work	`curl /api/projects`
6	**Seeder + asset generator** (images, videos, long HTML with aligned figures)	Real-looking data makes every later screen honest	`db:seed` fills everything
7	**Admin foundation**: login, Sneat layout, menu, navbar	Everything in admin needs it	Dashboard opens
8	**Generic CRUD engine** → Projects, Posts, Categories, Skills, Services, Timeline, FAQs, Colours	One engine, eight screens	Create / edit / delete works
9	**Rich editor + uploads + media library**	The hardest UX; needs real data to test	Image aligned left in a post
10	**Contact pipeline**: DB → mail → Google Sheet → Excel + admin inbox	Business-critical; failure must not lose a message	Message lands in 4 places
11	**Settings + theme colours + live search + dashboard**	Polish the owner experience	Colour added in admin shows on site
12	**Ported the design to React**: contexts → layout → pages → animations	Now the API is stable, the port is mechanical	Every route renders from API
13	**Tested** (API, admin, React render, MySQL) and fixed 9 real bugs (chapter 9)	Before docs, because docs must describe truth	All checks green
14	**Documentation + packaging** (these PDFs, zip)	So *you* can run and extend it alone	This book

Banglay

Design -> content list -> environment -> database -> API -> demo data -> admin -> editor -> contact -> settings/search -> React -> test -> docs. Protita dhap e ekta "chalano jay emon jinish" paoa jay — eta-i project ke bhoy-hin rakhe.

4. Key decisions (and the alternatives I rejected)

Decision	Why	Rejected alternative
Two apps (React + Laravel API) instead of Blade-only	You asked for React; the API can later feed a mobile app too	Inertia / Blade: simpler but not what you wanted
Admin inside Laravel (Blade + Sneat)	Sessions, validation, uploads are native; Sneat gives a finished UI	Admin in React: double the work, needs token auth
Generic CRUD controller driven by field arrays	8 resources in ~40 lines each; every new resource is 5 minutes	Filament/Nova: heavy, different look than Sneat; hand-written controllers: 8× code
Settings schema in `config/site.php`	Add a field in one array → admin form + API + defaults appear	One column per setting: migration for every tweak
TinyMCE (self-hosted) editor	Image upload, captions, tables, code, media, alignment *classes*, no API key	CKEditor 5 (licence key needed), Quill (weak images)
Alignment via CSS classes (`align-left/right/center`)	Same classes style editor and website; survives design changes	Inline `style="float:left"`: ugly, hard to restyle
Store image paths relative (`/storage/…`), API rewrites to current host	Move domain / go live without editing content	Absolute URLs stored in text: break on move
Contact = DB first, then side-effects (mail, sheet) in try/catch	A mail outage must never lose a lead	Send mail only: no history
Google Sheet through Apps Script webhook	No Google Cloud project, no OAuth, 5-minute setup	Sheets API + service account: stronger but complex
Excel with a tiny built-in writer	Zero composer packages to break on shared hosting	maatwebsite/excel: powerful, heavier
CSS 3D + JS for the ring, not Three.js	3 KB of logic, touch-friendly, no WebGL fallback issues	Three.js: 600 KB for a carousel
Lenis + GSAP ticker for smooth scroll	ScrollTrigger stays perfectly in sync	Native `scroll-behavior`: no inertia
Real URLs (`/projects/slug`) with React Router	Shareable links, back button, SEO-friendlier	Hash routes: fine for prototypes only
Motion toggle + `prefers-reduced-motion`	Accessibility and low-end phones	Always-on animation

5. Data model

The database is the project. Time spent here is repaid in every later step.

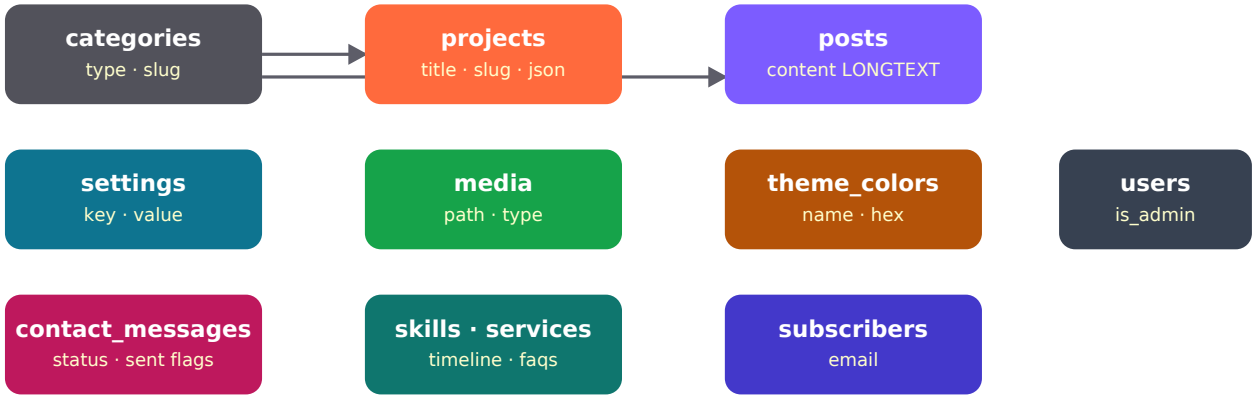


Table	Key columns	Design note
`projects`	slug, title, category_id, cover_image, hero_video, **tags/results/features/gallery (JSON)**, overview/challenge/solution/**content (LONGTEXT)**, is_published, is_featured, sort_order	Short lists → JSON columns (no join tables). Big stories → LONGTEXT HTML
`posts`	slug, title, excerpt, cover_image, **content LONGTEXT**, tags JSON, read_time, views, published_at	`read_time` computed on save; future `published_at` hides a post
`categories`	type (project/blog), name, slug	One table for both kinds
`settings`	key (unique), value	Field list lives in `config/site.php`, not in the DB
`theme_colors`	name, hex, is_default, sort_order	The accent palette for the whole site
`media`	path, type, mime, size, width, height	Every upload is registered (library + safe deletion)
`contact_messages`	name, email, project_type, budget, message, status, mail_sent, sheet_synced, note	Flags show what succeeded
`skills`, `services`, `timeline_items`, `faqs`, `subscribers`	small content tables	sort_order everywhere for manual ordering
`users`	standard + `is_admin`	No roles package needed for a single owner

Seven design rules I applied

1. **Slug + published flag** on every public thing (clean URLs, drafts).
2. `sort_order` instead of guessing order by id.
3. **JSON columns** for small lists you never query inside (tags, features).

4. **LONGTEXT** for anything an editor can make big.
5. **Store paths, build URLs at output** (`Media::url()`), so storage can move to S3 later.
6. **Flags for side-effects** (`mail_sent`, `sheet_synced`) — retry later.
7. **One migration per feature area** while the project is young; split later.

6. API design

Contract-first. Before writing a controller I wrote the JSON each page needs. The key idea is the **bootstrap** call: instead of ten small requests when the site opens, one `GET /api/bootstrap` returns settings, colours, services, skills, timeline, FAQs and categories. Lists (`/projects`, `/posts`) return *card-sized* objects; detail endpoints return the heavy fields (HTML, gallery, related items).

```
// card shape (list)                          // detail adds
{ "slug": "bluebee-platform",                   "overview": "<p>…</p>", "content": "<h2>…</h2>…",
  "title": "BlueBee Platform", "code": "Bb",     "gallery": [{"src": "http://…/p.jpg", "caption": ""}],
  "category": "web", "cover": "http://…jpg",     "video": "http://…/demo.mp4", "video_kind": "file",
  "tags": ["Laravel","MySQL"], "featured": true }  "next": { "slug": "…", "title": "…" }
```

Rules

- **Nouns, plural, lowercase** (`/projects`), slug for detail (`/projects/{slug}`).
- **Always JSON errors:** 422 with `errors`, 404 with `message`. The React `api()` helper turns them into exceptions.
- **Never expose secrets:** `config/site.php` marks fields `public => false` (contact e-mail, sheet URL/secret). `Setting::publicMap()` filters them.
- **Abuse protection:** `throttle:contact` (5/min/IP) + hidden honeypot input `website`.
- **CORS** is explicit (`FRONTEND_URL`). Images read by `<canvas>` need CORS too — hence `/api/image/{path}`.
- **No pagination yet** (a portfolio has dozens, not millions). Add `->paginate()` + `?page=` when lists grow — the React code only needs a "load more" button.

> **Banglay**
>
> API = site er "waiter": React jeta chay shetai dey. Prothome JSON er chehara thik kori, tarpor code. Secret jinish (mail, sheet URL) kokhono API te pathai na.

7. The admin dashboard

7.1 Sneat inside Laravel

Sneat is plain HTML/CSS/JS. Laravel does not need it converted — I copied its `assets/` folder to `public/sneat/assets` and rebuilt its page frame **once** as Blade: `layouts/app.blade.php` (head, scripts, toasts), `partials/menu.blade.php` (sidebar), `partials/navbar.blade.php` (search + user menu). Every other page just writes `@extends('admin.layouts.app')` and fills `@section('content')`. The frame is a template; the pages are data.

7.2 The generic CRUD engine

A resource is described, not programmed. The controller returns a list of **fields**; the engine builds the form, validates, saves uploads and renders the table. This is the single most valuable pattern in the project.

```
class SkillController extends CrudController {
  protected function model(): string { return Skill::class; }
  protected function meta(): array { return ['title'=>'Skills','singular'=>'Skill','route'=>'skills','icon'=>'bx
      -bulb']; }
  protected function columns(): array { return [['key'=>'name','label'=>'Skill','type'=>'title'],
      ['key'=>'level','label'=>'Level %']]; }
  protected function fields(): array { return [
     ['name'=>'name',  'label'=>'Skill name','type'=>'text',   'col'=>6, 'rules'=>'required|max:80'],
     ['name'=>'level', 'label'=>'Level 0-100','type'=>'number','col'=>3,
      'rules'=>'required|integer|min:0|max:100'],
  ]; }
}
// routes/web.php:  Route::resource('skills', SkillController::class)->except('show');
```

Field type	Renders	Saved as
`text` `textarea` `number` `select` `color` `datetime` `toggle`	Bootstrap inputs	plain column (numbers coerced; empty → default)
`image`	file input + live preview + "remove" tick	path on the public disk, old file deleted
`video`	file upload **or** a pasted YouTube/Vimeo/mp4 URL	path or URL
`gallery`	many images at once; per-image caption + Keep tick	JSON `[{src, caption}]`
`editor`	TinyMCE rich text	HTML
`tags` / `list`	comma separated / one per line	JSON array
`section`	a card heading to group fields	—

7.3 The rich editor — how "image anywhere, any alignment" works

1. TinyMCE lets the author put an image on any line: **start**, **middle**, **end**. Image Upload → `POST /admin/media/editor-upload` → file stored + registered in the media library → editor receives `{location: "/storage/editor/…jpg"}`.
2. Alignment buttons are re-mapped (in the `formats` option) to add **CSS classes** — `align-left`, `align-center`, `align-right` — instead of inline styles. Size classes: `img-full`, `img-md`, `img-sm`. Captions use `<figure class="image"><figcaption>`.
3. The same rules exist twice: in the editor's `content_style` (so you see the final look while writing) and in the website's `.prose` CSS. Left/right = `float` with margin so text wraps; centre = block with auto margins.
4. The database stores **relative** `/storage/…` URLs. `Media::fixHtml()` rewrites them to the current API host on output, so moving to a new domain does not break old articles.

```
.prose .align-left  { float:left;  margin:6px 28px 16px 0; max-width:46%; }
.prose .align-right { float:right; margin:6px 0 16px 28px; max-width:46%; }
.prose .align-center{ display:block; margin:0 auto; float:none; clear:both; }
```

7.4 Live search, settings, messages

- **Live search** (`/admin/search?q=`): debounce 160 ms → `SearchService::admin()` runs small `LIKE` queries on 8 tables → JSON with `title, sub, type, url, image` → dropdown with the thumbnail on the right and keyboard navigation. The public site uses the same idea at `/api/search`.
- **Settings:** `config/site.php` lists groups and fields; `SettingsController` builds tabs from it. Values live in the `settings` table; missing values fall back to the schema default; the whole map is cached and cleared on save.
- **Messages pipeline:** save → mail (try/catch) → sheet (try/catch) → flags → optional auto-reply. Failure of a side-effect is logged, never shown to the visitor, and visible in the inbox columns.
- **Animations in admin:** GSAP staggers cards/rows/menu in, counters count up, toasts slide and auto-dismiss, hover lifts on cards — all in `admin-assets/admin.js`/`admin.css`.

8. The React frontend

8.1 Structure

Folder	Contains	Idea
`src/context`	Site, Prefs, Smooth, UI, Transition	Global state, each in its own provider
`src/hooks`	`useApi`, `useGsap` (+ helpers)	Reusable behaviour
`src/components`	Navbar, overlays, Ring3D, HeroCanvas, VizCanvas, PointerFX…	UI building blocks
`src/pages`	Home, About, Projects, Blogs, Contact, 2 details, 404	One file per route
`src/styles.css`	tokens + all component CSS	Design tokens: `--bg --fg --a`

8.2 Provider stack (outer → inner)

```
<BrowserRouter>
  <SiteProvider>        // GET /api/bootstrap once; shows "Backend not reachable" if it fails
    <PrefsProvider>     // theme, accent, font, motion, cursor, sound  → <html data-theme style="--a">
      (localStorage)
      <SmoothProvider>  // Lenis, wired into gsap.ticker
        <UIProvider>    // which overlay is open: menu / search / settings
          <TransitionProvider>  // go(path): 3D curtain → navigate()
            <App/>
```

Why this order? Prefs needs site data (default accent comes from the admin colours). Smooth needs prefs (motion off = no Lenis). Transition needs everything above because it closes overlays, scrolls to top and respects motion.

8.3 Theming

Colours are CSS variables. `PrefsProvider` writes `--a` (accent), `--ai` (readable text on accent, computed from luminance), `data-theme` and `data-ft` on `<html>`. Nothing in components knows a hex value — add a colour in admin and the whole site follows. A visitor's own choice is saved in `localStorage` and wins over the admin default.

8.4 The animation catalogue

Effect	Technique	Where
Letter-by-letter 3D title flip	`<SplitChars>` wraps letters; `gsap.from(.c3, {rotationX:-100, stagger})`	page titles
Word-by-word scroll reveal	`<SplitWords>` + ScrollTrigger `scrub:true`	Home intro, About lead

Effect	Technique	Where
Section reveal with 3D tilt	`ScrollTrigger.batch(.rv)` from `y:60, rotationX:10`	all pages
Count-up numbers	tween an object, write `textContent` on update	stats
Marquee reacting to scroll speed	rAF loop; scroll velocity changes speed, direction and skew	Home
Hero scroll fade / parallax	timeline with `scrollTrigger:{scrub:1}`	Home
Pixel portrait that explodes sideways	canvas particles; `scrollY` → eased progress → x offset ±, alpha ↓	Home hero
Canvas "viz" (dots / squares / rings / network)	rAF + cursor influence; mode switched by tabs	Home
3D carousel	cards `rotateY(i·360/n) translateZ(R)`; ring `rotateY(rot)`; drag + inertia	Projects, Blogs
Tilt cards + magnetic buttons + cursor	one document-level `mousemove` handler (event delegation)	everywhere
Page transition curtain	7 slats flip with `rotationX` (3D), page name flips in, `navigate()` runs mid-way	every route change
Smooth scroll	Lenis `raf` called from `gsap.ticker`; `lenis.on("scroll", ScrollTrigger.update)`	global

8.5 The pattern that keeps GSAP safe in React

```
const ref = useGsap((root) => {            // runs only when Motion is ON
  gsap.from('.title .c3', { rotationX: -100, stagger: .03 });
  ScrollTrigger.batch('.rv', { onEnter: b => gsap.to(b, { opacity: 1, y: 0 }) });
}, [dataThatChangesTheDOM]);
return <div ref={ref}>…</div>;               // selectors are scoped to this element
// inside useGsap:  const ctx = gsap.context(fn, ref);   return () => ctx.revert();   // undo on unmount
```

Rules: (1) always create animations inside `gsap.context` and `revert()` on cleanup — otherwise triggers pile up when you navigate; (2) never animate elements React may re-create without re-running the hook (that is the `deps` array); (3) animation must be **optional** — with Motion off the page is complete and readable; (4) after route change call `ScrollTrigger.refresh()` once content/images have their final height.

8.6 How the hero portrait works

1. Draw the photo (or a vector placeholder) onto an off-screen canvas, grayscale.
2. Sample it into a grid (5 px cells). Every bright cell becomes a **particle** `(x, y, brightness, random1, random2)`.
3. Every frame draw each particle as a thin bar. Opacity = brightness × *reveal*, where reveal is high near the cursor and low elsewhere; near the cursor rows also wobble sideways (glitch).
4. `progress = clamp(scrollY / 0.85·viewportHeight)` (eased). Each particle moves **left or right** depending on its x, by `progress² × random`; also drifts vertically and fades. Scroll back up →

progress → 0 → they re-assemble.

5. The image must be CORS-readable to sample pixels. The API therefore serves it through `/api/image/…` with CORS headers; if sampling fails the code falls back to the placeholder instead of crashing.

8.7 Performance and accessibility

- Canvas loops skip work when off-screen; ≤ ~7 000 particles; DPR capped at 1.5.
- Lenis + GSAP share one ticker; listeners are removed in `useEffect` cleanups.
- `prefers-reduced-motion` and the MOTION toggle disable GSAP, Lenis and canvases motion — pages stay complete.
- Real `<a href>` links (even with the curtain) so middle-click / open-in-new-tab / SEO work. Search overlay has keyboard navigation (↑ ↓ Enter Esc).

9. Testing — and the 9 real bugs it found

I did not "run it once and hope". Four layers of checks, each automated, each re-run after every fix.

Layer	How	Checked
Public API	Python script against `php artisan serve`	bootstrap, lists, detail, search, contact (valid + invalid), subscribe, 404, CORS header, image CORS, path traversal → 404
Admin	Python session (login + CSRF)	24 pages render; create project with cover + video + 2 gallery files + 24 KB HTML; update (gallery remove, YouTube URL); delete; post create; editor upload; media upload; settings → API; colour → API; Excel opens in a spreadsheet library; CSV; message status; sheet sync error message; logout
Google Sheet	mock Apps Script server	payload shape, secret accepted / rejected → returns true / false
React app	bundle + jsdom + real Laravel API	shell, hero from API, stats, work rows, 3D rings (6 cards), grid view, search dim, project detail (video, gallery), blog detail (3 aligned figures, TOC), contact submit, live search with images, settings drawer, accent/theme/motion toggles, 404; zero console errors
Database	same suites on **MySQL (MariaDB)** and SQLite	migrations, JSON columns, seeds, queries

#	Bug	Cause	Fix / lesson
1	`/admin` returned 404	A real folder `public/admin` shadowed the route	Renamed assets to `admin-assets`. *Never name a public folder like a route.*
2	Saving a project failed: NOT NULL `sort_order`	Empty number input → `""` → Laravel turns it into `null`	Coerce numbers: empty → field default. *Normalise input at the boundary.*
3	Editing a project changed its URL	Empty slug field regenerated from the new title	Keep the existing slug on update. *URLs are promises.*
4	Update silently unpublished an item	Unchecked checkbox sends nothing → `false`	Hidden `0` input before the checkbox (done in the form). *Checkbox ≠ boolean.*
5	Hero photo could not be read by canvas	Cross-origin image taints the canvas (`getImageData` throws)	CORS-enabled `/api/image/…` + graceful fallback. *Pixels need CORS.*
6	Old articles would break after moving domain	Absolute URLs stored inside HTML	Store `/storage/…`, rewrite on output. *Store relative, render absolute.*
7	Console error after leaving Home	rAF callback ran once after unmount (ref was null)	Null-guard refs inside loops. *Cleanup is not instant.*

#	Bug	Cause	Fix / lesson
8	Whole prototype hidden under the loader	A CDN script failed → later code never ran → preloader never removed	Final React app bundles dependencies; preloader/curtain have fail-safes. *A decorative layer must never block the site.*
9	Preloader counter crashed after fail-safe removed it	Animation updated a deleted element	Guard element lookups in tween callbacks. *Animations must be null-safe.*

10. Four flows, step by step

A. A visitor opens a project

1. Browser loads `index.html` → React starts → `SiteProvider` calls `GET /api/bootstrap`.
2. Router matches `/projects/bluebee-platform` → `ProjectDetail` calls `GET /api/projects/bluebee-platform`.
3. `PublicController::project` loads the model + category, builds `toApi(true)`: media paths → URLs, HTML run through `Media::fixHtml`, next project added.
4. React renders meta, tilt cover, `<video>`, prose HTML, gallery; `useGsap` runs the entrance animations.

B. A visitor sends a message

1. `Contact.jsx` posts JSON to `/api/contact` (throttled, honeypot checked).
2. Validation (422 → shown under the form). `ContactMessage::create`.
3. `Mail::to(recipients)->send(ContactReceived)` with *Reply-To = visitor*. `GoogleSheet::push()` → Apps Script appends a row.
4. Flags saved; optional auto-reply; JSON `{ok:true}` → success box. You see it in Admin → Messages (+ Excel download).

C. You write a blog post with images

1. Admin → Posts → New → fill fields → in the editor insert images (upload → library) and align them.
2. Save → `PostController::beforeSave` computes `read_time`, sets `published_at` → `Post::create`.
3. Website `/blogs` and the 3D ring show it immediately; `BlogDetail` builds the table of contents from the `<h2>` tags.

D. Live search

1. Visitor presses `/` → overlay opens, input focused.
2. Typing → 180 ms debounce → `GET /api/search?q=` (previous request aborted).
3. `SearchService::public` queries projects and posts, scores title matches first → JSON with image URL → rows with the thumbnail on the right → Enter opens the page via the curtain transition.

11. Recipes: extend the project

Recipe 1 — add "Testimonials" (30 minutes)

```
$ php artisan make:model Testimonial -m          # 1 columns: name, role, quote, avatar, sort_order
$ php artisan migrate
# 2 app/Http/Controllers/Admin/TestimonialController.php  → copy SkillController, change fields (text, text,
      textarea, image)
# 3 routes/web.php:  Route::resource('testimonials', A\TestimonialController::class)->except('show');
# 4 resources/views/admin/partials/menu.blade.php: one line in $items
# 5 PublicController::bootstrap(): 'testimonials' => Testimonial::orderBy('sort_order')->get()
# 6 React: const { testimonials } = useSite();  map them in a <section> on About
```

Recipe 2 — more

Goal	Do this
New site setting	add a field to `config/site.php` (group, key, label, type, default) → it appears in admin and `settings.key` in React
New page in React	file in `pages/`, `<Route>` in `App.jsx`, link in `Navbar.jsx`/`MenuOverlay.jsx`
New admin menu item	one line in `partials/menu.blade.php`
Change default colour	Admin → Theme colours → tick "Default accent"
Multi-language	add `locale` column + `/api/...?lang=` and a language switch in `PrefsProvider`
Pagination	controller `->paginate(12)`; React "Load more" using `?page=`
Analytics	add the script tag in `frontend/index.html`
Cache heavy API	`Cache::remember` in `PublicController::bootstrap`, clear in `Setting::forgetCache()`

12. The Playbook — if you build another full-stack project

Print this chapter. It is the method, with no project-specific details.

Phase	Typical time*	Checklist	Done when
0 Discovery	0.5–1 day	goal sentence · users · must/should/later features · examples they like · deadline · budget · who gives content	client can read your 1-page summary and says "yes"
1 Design	2–5 days	sitemap · style references · 1 clickable HTML page of the hardest screen · light/dark? responsive? languages?	design approved in writing
2 Data	0.5–1 day	entities · relations · slugs/published/sort · JSON vs tables · seed ideas	migration runs; ER sketch
3 API	1–2 days	endpoints list · JSON shapes · validation rules · errors · CORS · rate limits	every endpoint answers in `curl`
4 Seed	0.5 day	realistic long texts, 20+ images, awkward cases (very long title, no image)	demo site feels real
5 Admin	2–4 days	login · generic CRUD · rich editor · media · settings · messages · search	owner can run the site alone
6 Frontend	4–8 days	tokens · layout · pages · loading/empty/error states · motion · responsive pass	works on a 360 px phone and 1920 px screen
7 QA	1–2 days	automated API/admin checks · browser pass · MySQL run · accessibility pass · reduced-motion	no console errors; checklist green
8 Ship	0.5–1 day	env · HTTPS · backups · mail · monitoring · handover doc + video	client logged in and edited something
9 Care	ongoing	monthly updates · backup test · analytics review	restore test passed

*Rough solo-developer estimates for a portfolio-size project; scale for complexity.

Ten rules I always follow

1. Make it **run** first, **right** second, **pretty** third.
2. Never wait more than an hour to run what you wrote.
3. Write the JSON shape before the controller; the SQL before the model.
4. Everything repeated three times becomes a generic piece (CRUD engine, settings schema).
5. Side-effects (mail, sheets, webhooks) go **after** saving and inside try/catch.
6. Validate on the server, always; the browser is only courtesy.
7. Keep secrets in `.env`, never in code or API output.
8. Decoration must be **optional** (motion off still works).
9. Test with ugly data: 200-character titles, no images, emoji, Bengali text.
10. Document while it is fresh — future you is a stranger.

Questions to ask the client in Discovery

- What decision should a visitor make after 30 seconds? What proof do they need?
- Who updates content, how often, and are they technical?
- Three sites you love and *what exactly* you love in each.
- What must be measurable (leads, sign-ups)?
- What can wait for version 2?
- Hosting, domain, e-mail already owned? Deadline reason?

Common mistakes

- Starting with the home-page animation before any data exists.
- One mega-table or hand-written CRUD for every entity.
- Absolute URLs inside stored content.
- No fallback when the API is down or a photo is missing.
- Skipping admin ("we will edit the database") — then no one can maintain it.
- Testing only the happy path.
- Adding features while the previous one is untested.

Git and delivery rhythm

```
$ git init && git add . && git commit -m "chore: laravel skeleton + env example"
# one commit per step of chapter 3, e.g.:
# "feat(db): portfolio tables"  "feat(api): public endpoints"  "feat(admin): generic crud"  "feat(web): home
      page"
$ git tag v1.0.0
```

Appendix

Glossary (Banglish)

Term	Mane
API	Website r server er moddhe kotha bolar niyom (JSON dey)
Migration	Database table banano/bodlano-r code
Seeder	Demo data automatic bosano
CRUD	Create / Read / Update / Delete
CORS	Browser er nirapotta niyom — onno domain theke data nite hole server ke "ha" bolte hoy
SPA	Single Page App — page reload chara bodlay (React)
Slug	URL er sundor naam: `my-first-post`
Scrub	Animation scroll er sathe sathe cholbe
Context (React)	Sob component er jonno shared data
Hook	React er function jeta state/effect dey
Debounce	Typing thamle tarpor request pathano
Throttle	Ekta somoye maximum koyta request — spam □□□□□□
Honeypot	Bot ke dhorar lukano field
Webhook	Ekta URL jekhane data push kora hoy (Google Sheet)
LONGTEXT	Boro text rakhar column type

Command cheat sheet

```
# backend
composer install            php artisan portfolio:install [--fresh]      php artisan serve
php artisan migrate         php artisan db:seed                          php artisan storage:link
php artisan config:clear    php artisan route:list                       tail -f storage/logs/laravel.log
# frontend
npm install   npm run dev   npm run build   npm run preview
# production
composer install --no-dev --optimize-autoloader   php artisan migrate --force   php artisan config:cache
      route:cache view:cache
```

Where to go next

- Add roles (editor vs admin) with `spatie/laravel-permission` — you already use this pattern in your other projects.

- Move uploads to S3 / Cloudflare R2 (`FILESYSTEM_DISK=s3`) — the code already builds URLs from paths.
- Server-side rendering / pre-rendering (Next.js or `vite-plugin-ssr`) if SEO becomes critical.
- Queue the e-mail + sheet push (`QUEUE_CONNECTION=database` + `php artisan queue:work` under Supervisor).
- Add automated tests with `php artisan test` (Pest/PHPUnit) using the scripts in this book as a starting list.